



RIPHAH INTERNATIONAL UNIVERSITY LAHORE

ASSIGNMENT 03

Name: Kanwal Naeem

SAP ID: 58803

Class: BSCS-5A

Session: Spring 2026

Course: Game Programming

Submitted to: Mr. Hasham Zahid

Code Architecture Design: Apex Legends

1. Game Overview

Apex Legends is a multiplayer battle royale game where 20 squads of 3 players fight until only one team survives.

Core gameplay loop:

- Players are grouped into squads
- They drop into a large map from a dropship
- They loot weapons, armor, and supplies
- They fight other squads using guns + unique character abilities
- The playable area shrinks over time (ring system)
- Last surviving squad wins

Why this matters architecturally:

This is not just a “game,” it is a real-time distributed simulation system where:

- Hundreds of entities move simultaneously
- Network latency is a constant enemy
- Fairness must be enforced centrally (server authority)

2. High-Level Architecture (Major Systems)

Think of the game as 6 big systems working together like a machine that *barely stays stable under pressure*.

2.1 Player System

Responsibility:

Handles everything about a player's identity and state.

Includes:

- Login / authentication
- Player stats (kills, wins, rank)
- Character selection (Legend choice)
- Persistent progression data

Why it exists:

Without this, every match would treat you like a random ghost instead of a real account.

2.2 Matchmaking System

Responsibility:

Builds fair matches and assigns servers.

Workflow:

1. Player queues into matchmaking
2. System groups players by:
 - Skill rating (MMR)
 - Region (latency)
 - Party size

3. Forms squads of 3

4. Assigns a game server instance

Why it matters:

Bad matchmaking = angry players = dead game.

2.3 Game State Management (Core Brain)

Responsibility:

Keeps the entire match synchronized.

Runs on:

- Server tick loop (e.g., 20–60 ticks per second)

Manages:

- Player positions
- Zone (ring) shrinking
- Projectile simulation
- Ability effects
- Match rules (win/loss conditions)

Key idea:

The server is the single source of truth. Clients only *suggest*, server decides.

2.4 Combat System

Responsibility:

Handles all fighting mechanics.

Subsystems:

- Hit detection (raycasts / hitboxes)
- Damage calculation
- Armor and shield logic
- Ability effects (grenades, ultimates)
- Cooldowns

Important design rule:

Client may *predict*, but server always *verifies*.

2.5 Inventory System

Responsibility:

Manages everything players pick up.

Includes:

- Weapons
- Ammo
- Attachments
- Healing items

Behavior:

- Loot spawns in world
- Players pick items → server validates pickup
- Inventory updates replicated to all relevant clients

2.6 UI System

Responsibility:

Shows everything to the player.

Includes:

- Health bars
- Ammo count
- Map / ring indicator
- Kill feed
- Menus

Key idea:

UI is NOT authoritative. It is just a visual layer.

3. Code Structure (How the code is actually organized)

Now we stop talking abstract and start talking like engineers.

3.1 Folder Structure

```
/client
  /rendering
  /input
  /prediction
  /ui

/server
  /game_logic
  /match_manager
  /combat
  /anti_cheat

/shared
  /models
  /network_messages
  /constants
```

3.2 Core Classes & Responsibilities

PlayerController (Client)

- Reads player input
- Sends movement commands to server
- Performs prediction to reduce lag feel

PlayerEntity (Server)

- Stores real player state
- Validates movement and actions
- Prevents cheating

MatchManager (Server)

- Starts match
- Assigns players to squads
- Ends match when win condition is met

LegendAbilitySystem

- Defines character abilities (e.g., teleport, shields)
- Handles cooldowns + effects
- Ensures balance rules are enforced server-side

InventoryComponent

- Attached to each player
- Stores weapons/items
- Syncs with client UI

3.3 Communication Flow

Client → Server:

- Input commands (move, shoot, use ability)

Server → Client:

- World updates
- Other players' positions
- Combat results

Networking style:

- UDP-based communication
- Delta compression (only changes are sent)
- Snapshot interpolation (smooth movement)

3.4 Simple Architecture Diagram

```
[Client]
|
|  input commands (UDP)
v
[Server Simulation]
|
|  validated state updates
v
[All Clients]

Server contains:
- Game State
- Combat Logic
- Match Rules
- Anti-cheat validation
```

4. Design Decisions (Why this architecture exists)

Now the part where everything sounds “obvious” after you already built it.

4.1 Server-Authoritative Model

Why:

Because if clients control the game:

- Everyone cheats instantly
- Game becomes a meme simulation

Benefit:

- Fair gameplay
- Cheat resistance
- Consistent world state

4.2 Component-Based Design

Instead of giant monolithic classes:

You get:

- InventoryComponent
- MovementComponent
- AbilityComponent

Why it matters:

- Easier to add new Legends
- Easier debugging
- Better scalability

4.3 Client Prediction + Server Reconciliation

Problem:

Network lag makes movement feel delayed

Solution:

- Client predicts movement instantly
- Server corrects mistakes later

Result:

Smooth gameplay that doesn't feel like a slideshow

4.4 Shared Code Layer

Why:

Avoid duplicating logic on client and server.

Example:

- Damage formulas
- Ability definitions
- Item data